Prompt2Model: Language-Guided Vision Model Factory

Deep Research Report and Implementation Blueprint

Authors: Dev Sanghvi, Venkata Sai Aneesh Thatiparti, Madhuvani Thatiparti

Date: April 2026

Scope, Success Criteria, and Precise Problem Definition

The traditional computer vision training pipeline requires continuous human intervention and profound domain expertise to navigate complex hyperparameter landscapes, optimal architecture selections, and robust data augmentation strategies.[1] This manual calibration introduces significant friction for rapid prototyping and creates formidable barriers to entry, particularly when engineering models bound by strict latency and memory constraints for edge computing deployments.[3] The fundamental problem this project addresses is the abstraction of these expert engineering choices through a natural language interface, converting user intent directly into a mathematically constrained optimization problem. The core objective is the development of a lightweight, Language-Guided Vision Automated Machine Learning (AutoML) system—a localized "model factory." This system is engineered to ingest a natural language task specification alongside an arbitrary vision dataset, autonomously configure a constrained training pipeline, execute a strictly budgeted model tuning loop, and yield a self-contained, deployable model package accompanied by an exhaustive evaluation report.[5]

The system architecture enforces a strict operational contract. The inputs are confined to a natural language specification dictating the task type, target label synonyms, optimization constraints reflecting a speed-versus-accuracy tradeoff, and environmental data assumptions, coupled with an image dataset formatted as classification directories or COCO-standard JSON annotations.[8] The outputs are restricted to an Open Neural Network Exchange (ONNX) model artifact embedding all necessary pre-processing and post-processing logic, alongside a comprehensive analytical report detailing the model's predictive performance and resource utilization profile.[9]

Success is not defined merely by training a functional neural network, but by the autonomous execution of the entire pipeline under a fixed computational budget. The success criteria are divided into three primary vectors. The first is Language-to-Configuration Correctness, which measures the fidelity of the translation from natural language to pipeline hyperparameters. This is evaluated using a manual rubric applied across twenty to thirty diverse prompts, quantifying whether specific environmental descriptors correctly trigger corresponding augmentation operations and whether constraint directives accurately influence the architectural backbone selection. The second criterion is Budgeted Vision Performance, ensuring the selected models achieve competitive classification accuracy, F1 scores, or Mean Average Precision (mAP) on

held-out validation splits without exceeding the predetermined GPU-hour allowance.[12] The final criterion is Operational Deployment Viability, demonstrating that the output artifacts successfully encapsulate both the neural weights and the associated inference metadata, enabling zero-configuration deployment on edge runtimes.[10]

System Architecture Blueprints

The Prompt2Model architecture operates as a sequential, multi-agent orchestration pipeline that bridges large language models with discrete computer vision training operations. The pipeline systematically decomposes the user's unstructured request into executable machine learning operations.[15]

The first stage is the Spec Parser and Constraints engine. This module utilizes a foundational large language model tasked with extracting actionable configuration parameters from the user's natural language input. To prevent hallucination and ensure output consistency, the language model is wrapped using the instructor library, which enforces strict adherence to a predefined pydantic schema.[16] The extracted schema dictates the target computer vision task, compiles a list of user-requested labels, parses latency versus accuracy preferences into a normalized continuous variable, and identifies environmental lighting or weather constraints.[19]

Following configuration extraction, the pipeline initiates the Label Resolver and Vision Dataset Handler. Users frequently describe targets using colloquial synonyms that do not perfectly match the rigid class names present in the provided dataset annotations. To reconcile this discrepancy autonomously, the system deploys a zero-shot Contrastive Language-Image Pretraining (CLIP) mechanism.[20] By encoding both the user-provided synonyms and the dataset's ground-truth labels into a shared high-dimensional semantic space, the system computes a cosine similarity matrix. This matrix enables the autonomous mapping of the user's natural language taxonomy directly to the dataset's categorical structure, bypassing the need for manual dictionary mappings.[22]

The third operational stage involves the Recipe and Model Selector. The strictly typed parameters generated by the parsing engine are mapped to discrete computer vision libraries. Environmental constraints are deterministically linked to data augmentation operations using the hardware-accelerated Albumentations framework.[25] Simultaneously, the latency constraints dictate the selection of the vision backbone from a curated pool of lightweight architectures optimized for the specified task.[27]

The fourth stage constitutes Budgeted Training. To respect the strict compute limitations inherent in edge-focused or course-budgeted environments, the training loop is orchestrated by Ray Tune in conjunction with the Optuna hyperparameter optimization framework.[29] This subsystem leverages aggressive early-stopping schedulers to prune underperforming hyperparameter combinations rapidly, ensuring that the exploration of the search space completes within the user-defined GPU-hour budget.[29]

The terminal stage encapsulates Evaluation, Analysis, Export, and Packaging. The optimal model

weights identified during the training phase are evaluated against the held-out test data. The resulting PyTorch computational graph is subsequently exported to the ONNX format. Crucially, the module utilizes the ONNX metadata_props specification to inject the required input resolution, normalization statistics, and label mappings directly into the model file, yielding a self-contained artifact capable of immediate execution on downstream edge platforms.[9]

Model and Method Roster and Current Buildable Landscape

The feasibility of this autonomous factory relies on a highly curated, constrained pool of state-of-the-art architectures and frameworks published between 2024 and 2026. The following table delineates the primary methods and models integrated into the Prompt2Model pipeline, justifying their inclusion based on execution viability and architectural efficiency.

System Component	Selected Architecture / Framework	Justification and Integration Strategy	Primary References
Constraint Parsing Engine	instructor integrated with pydantic	Enforces programmatic structure on large language model outputs. By defining the vision configuration search space as a pydantic class, the system guarantees that the extracted task parameters, augmentations, and constraints are inherently valid Python objects, eliminating string parsing errors and LLM hallucinations.	16
Zero-Shot Label Resolution	OpenAI CLIP (ViT-B/32 or ViT-L/14)	Provides zero-shot multimodal alignment. The text encoder generates dense semantic	20

		vectors for both user-requested object names and actual dataset labels. A cosine similarity threshold algorithm subsequently matches these vectors, bridging the semantic gap between natural language prompts and legacy datasets.	
Natural Language Augmentation	Albumentations	Serves as the high-performance backend for image transformations. Specific words extracted from the natural language prompt (e.g., "blurry", "dark") are deterministically mapped to specific functional calls (e.g., A.GaussianBlur, A.RandomBrightnessContrast), creating dynamic, condition-specific training recipes.	25
Lightweight Image Classification	MobileNetV3, EfficientNet-Lite, MobileViT	Deployed when the language parser detects a strict latency constraint. These architectures	4

		leverage depthwise separable convolutions and inverted residuals to maximize accuracy while minimizing computational FLOPs and memory footprint on edge hardware.	
Real-Time Object Detection	YOLOv10, YOLOv11, YOLOv12	Act as the primary single-stage detection backbones. The selection of recent YOLO variants is critical due to their NMS-free training architectures and attention-centric designs, which significantly streamline the ONNX export process and reduce end-to-end inference latency on edge accelerators like TensorRT.	[41]
High-Accuracy Object Detection	RF-DETR, RT-DETR	Vision transformer-based detection architectures reserved for scenarios where the natural language prompt	[41]

		explicitly prioritizes absolute mean Average Precision (mAP) over real-time execution speeds, leveraging multi-scale feature fusion without anchor boxes.	
Budgeted Hyperparameter Search	Ray Tune with Optuna Backend	Coordinates the training trials under strict time constraints. Optuna handles the define-by-run Bayesian search space optimization, while Ray Tune orchestrates the hardware resource allocation and applies the Asynchronous Successive Halving Algorithm (ASHA) for early termination of sub-optimal configurations.	[29]

Dataset Options and Preprocessing Strategy

To maintain operational autonomy, the Vision Dataset Handler is engineered to ingest standardized directory and annotation structures, circumventing the need for custom, dataset-specific dataloaders. For image classification tasks, the system expects a hierarchical directory structure where folder names denote class labels, or a flat directory accompanied by a comma-separated values (CSV) file mapping image identifiers to text labels. For object detection tasks, the system rigidly enforces the COCO JSON annotation format, which comprehensively encapsulates bounding box coordinates, class category mappings, and image dimensional metadata within a single, highly structured artifact.

The preprocessing strategy is fundamentally dynamic, driven by the constraints parsed from

the user's natural language input. A base preprocessing pipeline is universally applied to ensure tensor conformity, consisting of deterministic resizing operations tailored to the specific backbone's stride requirements (e.g., ensuring dimensions are multiples of 32 for YOLO architectures) and standard ImageNet statistical normalization.[45]

Beyond this base layer, the Albumentations framework constructs a customized sequential transformation pipeline based on the LLM's semantic extraction.[26] When the parser identifies environmental challenges in the prompt, such as "operate in poor weather," the system seamlessly injects operations like A.RandomRain and A.RandomFog into the training dataloader. Conversely, prompts emphasizing "variable lighting" trigger the inclusion of A.ColorJitter, A.RandomBrightnessContrast, and A.HueSaturationValue.[38] By mapping linguistic concepts directly to mathematical pixel transformations, the system ensures the resulting model develops robust feature representations tailored specifically to the user's anticipated deployment environment without requiring manual data engineering.[46]

Training Pipeline and Experiment Plan

The training pipeline is constrained by a predetermined, modest compute budget, necessitating a highly efficient exploration of the hyperparameter search space. The system bypasses exhaustive grid searches in favor of Bayesian optimization managed by the Optuna framework, seamlessly integrated within the Ray Tune orchestration engine.[30]

Upon initialization, the pipeline constructs a hyperparameter search space dictated by the selected model backbone. This space encompasses continuous variables such as learning rate boundaries and weight decay factors, alongside discrete choices regarding the application probabilities of the dynamically generated data augmentations.[31] The Ray Tune engine distributes these trial configurations across the available GPU resources. To enforce the computational budget, the Asynchronous Successive Halving Algorithm (ASHA) is employed. ASHA actively monitors the validation metrics of concurrent trials, aggressively terminating those configurations that fail to demonstrate competitive convergence in the early epochs. This mechanism concentrates the remaining computational budget exclusively on the most promising hyperparameter sets, allowing the system to output an optimized model within strict time limits (e.g., bounding total execution to a specific number of GPU-hours).[31]

Evaluation Framework and Metrics

The evaluation framework operates on two distinct analytical planes to validate both the algorithmic automation and the final predictive accuracy.

The first plane evaluates Language-to-Configuration Correctness. This assesses the reliability of the natural language parsing subsystem and the zero-shot label resolution. The framework executes an ablation suite against a static repository of twenty to thirty diverse edge-case prompts. The structured pydantic outputs and the resulting CLIP cosine similarity matrices are programmatically compared against a ground-truth expectation matrix. This yields a Parsing Accuracy metric, proving that the system correctly translates complex linguistic constraints

into accurate pipeline configurations without succumbing to large language model hallucinations or semantic matching errors.[34]

The second plane evaluates Budgeted Vision Performance and deployment efficiency. Following the completion of the Ray Tune optimization loop, the optimal model checkpoint is evaluated against a strictly held-out test split. For classification tasks, the system calculates overall accuracy and macro-averaged F1 scores to account for potential class imbalances. For object detection tasks, performance is quantified using Mean Average Precision at an Intersection over Union (IoU) threshold of 0.5 (mAP@0.5), as well as the more rigorous averaged mAP over multiple thresholds (mAP@[0.5:0.95]).[12] Simultaneously, the system profiles the architectural efficiency of the generated model, logging the total parameter count, theoretical floating-point operations per second (FLOPs), and an empirical measurement of inference latency reported in Frames Per Second (FPS) on the host hardware to verify adherence to the user's speed constraints.[51]

Runtime, Deployment, and Export Plan

The terminal objective of the model factory is the generation of an artifact that can be deployed effortlessly on edge devices. Traditional model export paradigms often fail in this regard, as they decouple the neural network weights from the requisite preprocessing logic and label dictionaries, requiring downstream engineers to manually reconstruct the data ingestion pipeline.[52]

The Prompt2Model system resolves this by leveraging the advanced metadata capabilities of the Open Neural Network Exchange (ONNX) format. The optimized PyTorch computational graph is exported using the torch.onnx.export utility. Following the structural export, the system utilizes the ONNX Python API to serialize the dynamic pipeline parameters directly into the graph's metadata_props dictionary.[32] Key-value strings representing the required input tensor resolution, the specific normalization means and standard deviations, and the finalized dataset label mapping dictionary are embedded immutably within the file.[54] This mechanism yields a completely self-contained deployment artifact. Edge platforms, such as NVIDIA Jetson devices running TensorRT, can programmatically parse this ONNX metadata to automatically configure their input streams and interpret the output logits, achieving true zero-configuration deployment.[9]

Failure Modes and Mitigation Strategies

The autonomous nature of the pipeline introduces specific systemic vulnerabilities that require explicit programmatic mitigations.

The primary failure mode involves Large Language Model Hallucinations during the specification parsing phase. The language model may attempt to request non-existent backbone architectures or generate malformed augmentation parameters. This is mitigated through the strict enforcement of pydantic schemas via the instructor library.[18] The schema enforces rigid data types and utilizes Literal constraints for model selection. If the language

model outputs invalid data, the system triggers an automated retry mechanism, feeding the validation error back to the model for correction, ensuring the vision pipeline only ever receives executable configurations.[19]

A secondary failure mode is Semantic Mismatch during label resolution. The CLIP model may fail to align highly specialized or ambiguous user synonyms with the dataset's ground-truth classes, potentially merging distinct categories or discarding valid labels. This is mitigated by establishing a strict cosine similarity threshold. If the similarity score between a user synonym and the dataset classes fails to exceed this threshold, the system autonomously falls back to deterministic substring matching or flags the specific label for manual review in the final evaluation report, preventing silent failures in the training data alignment.[56]

A final operational risk is Catastrophic Forgetting or Divergence during the budgeted hyperparameter optimization. Aggressive search spaces combined with limited training epochs can lead to models failing to converge. This is mitigated by establishing conservative boundary limits within the Optuna search space, enforcing learning rate warmups, and ensuring that the ASHA scheduler maintains a sufficient grace period before terminating early trials, guaranteeing that at least one stable configuration survives the optimization process.[1]

Reproducibility Checklist

To guarantee deterministic behavior and strict environmental consistency, the following reproducibility parameters are enforced across the pipeline:

- Establishment of a strictly pinned pyproject.toml or requirements.txt locking the versions of ultralytics, instructor, pydantic, ray[tune], optuna, and albumentations.
- Enforcement of fixed random seeds across the torch, numpy, and random libraries immediately prior to launching the Ray Tune execution engine.
- Utilization of a static benchmark suite of thirty standardized natural language prompts to consistently evaluate the parsing accuracy of the language-to-configuration module.
- Implementation of cryptographic checksums for the input dataset files to guarantee data integrity prior to initiating the training loop.
- Generation of a comprehensive config.json artifact alongside the ONNX model, redundantly logging all final hyperparameters and seed values utilized during the successful training run.

Week-by-Week Execution Plan

The project is structured across a rigorous twelve-week development cycle, culminating in a fully integrated system.

- **Weeks 1-2:** Core infrastructure initialization. Definition of the comprehensive pydantic schemas representing the computer vision search space and integration of the instructor language model client.
- **Weeks 3-4:** Implementation of the Natural Language Augmentation module mapping text to Albumentations, alongside the development of the CLIP-based zero-shot label

resolution engine.

- **Weeks 5-6:** Integration of the vision dataset handlers and the incorporation of the selected lightweight classification and detection backbones (e.g., EfficientNet-Lite, YOLOv11) into the PyTorch training loops.
- **Weeks 7-8:** Architecture of the Budgeted Training module. Wiring Optuna search spaces into the Ray Tune orchestrator and implementing the ASHA early-stopping logic.
- **Weeks 9-10:** Development of the export and packaging subsystem. Construction of the ONNX structural exporter and the metadata_props injection utilities to embed preprocessing constraints and label dictionaries.
- **Weeks 11-12:** Execution of the comprehensive ablation suite across the language and vision modules. Generation of the structured evaluation reports and preparation of the final integrated demonstration artifacts.

Prioritized Action Items

To ensure immediate momentum, the engineering team must prioritize the following executable tasks:

1. Finalize and lock the exact pydantic data structures that will serve as the immutable contract between the natural language parser and the downstream vision pipeline.
2. Prototype the CLIP cosine similarity matrix on a subset of the COCO dataset to calibrate the necessary confidence thresholds for successful zero-shot label mapping.
3. Deploy a minimal viable Ray Tune training loop utilizing a single lightweight backbone (e.g., YOLOv11n) to empirically validate the computational overhead and verify that the time-budgeting constraints function as intended.
4. Construct a standalone Python utility to inject and subsequently read metadata_props from a dummy ONNX file, confirming that downstream edge deployment tools can successfully extract the embedded label maps.

Open Bibliography and Code Links

The implementation relies heavily on the following foundational open-source repositories and architectural documentation:

- **Instructor (Structured LLM Output):** https://github.com/jxnl/instructor [17]
- **Pydantic (Data Validation):** https://docs.pydantic.dev/ [34]
- **OpenAI CLIP (Semantic Alignment):** https://github.com/openai/CLIP [57]
- **Albumentations (Image Transformations):** https://github.com/albumentations-team/albumentations [59]
- **Ultralytics (YOLO Backbones):** https://github.com/ultralytics/ultralytics [31]
- **Ray Tune and Optuna (Hyperparameter Optimization):** https://docs.ray.io/en/latest/tune/ [29]
- **ONNX Metadata Concepts:** https://onnx.ai/onnx/intro/concepts.html [32]

Prompt2Model: Language-Guided Vision Model Factory

Paper Analysis and Architectural Review

Executive Summary

The "Prompt2Model: Language-Guided Vision Model Factory" proposes a sophisticated Automated Machine Learning (AutoML) pipeline engineered to bridge the semantic gap between natural language intent and complex computer vision deployment. The project addresses the intense friction caused by the necessity for deep domain expertise in hyperparameter tuning, augmentation selection, and edge-device compilation. Designed for a 12-week, three-person execution timeline, the system ingests a user's natural language specification alongside a raw dataset, orchestrates a strictly budgeted training search, and outputs a standalone, metadata-rich ONNX model accompanied by a performance evaluation report. By integrating Large Language Models (LLMs) with high-efficiency vision backbones, the project democratizes access to custom, edge-ready computer vision solutions.

Problem Formulation and Scope

Developing performant vision models requires highly specific configurations that balance hardware constraints against predictive accuracy. The scope of this project is strictly defined to automate this configuration through linguistic extraction.

- **Inputs:** A plain-text task specification (detailing target labels, environmental conditions, and latency targets) coupled with an image dataset formatted as classification directories or COCO JSONs.
- **Outputs:** A production-ready ONNX model with embedded preprocessing rules, and a detailed markdown evaluation report highlighting metrics and system resource profiles.
- **Constraints:** The system operates under a hard computational budget, utilizing a constrained pool of pretrained backbones (2-3 per task) and limited hyperparameter search trials to ensure feasibility in resource-constrained environments. Supported operations are bounded to Image Classification and Object Detection, with Instance Segmentation designated as a stretch goal.

Architecture Deep-Dive

The system utilizes a linear, multi-agent pipeline to process requests:

1. **Spec Parser & Constraints:** Leverages an LLM wrapped in the instructor library to parse the user's natural language into a rigidly validated pydantic JSON schema, extracting explicit latency constraints and environmental strings.
2. **Label Resolver:** Implements OpenAI's CLIP model to perform zero-shot cosine similarity

matching. This aligns the colloquial label synonyms requested by the user with the discrete class annotations present in the dataset.

3. **Recipe Selector:** Maps the extracted environmental descriptors deterministically to hardware-accelerated image transformations using the Albumentations framework (e.g., mapping "poor lighting" to brightness and contrast jittering).
4. **Budgeted Training:** Orchestrates model training using Ray Tune and the Optuna optimization backend. This module enforces the strict computational budget by utilizing the Asynchronous Successive Halving Algorithm (ASHA) to aggressively terminate sub-optimal model configurations.
5. **Export & Packaging:** Exports the final optimized PyTorch graph to ONNX. Crucially, it serializes preprocessing requirements, statistical normalizations, and the finalized label map dictionary directly into the ONNX metadata_props.

Model and Method Landscape

The architecture synthesizes modern breakthroughs across artificial intelligence domains:

- **Language and Configuration:** instructor and pydantic eliminate prompt hallucination.
- **Semantic Bridging:** CLIP (ViT-B/32) provides the mathematical mechanism to map textual intent to visual categorization without manual intervention.
- **Vision Backbones:** Relies on highly optimized, modern architectures such as MobileNetV3 and EfficientNet-Lite for classification, alongside the NMS-free YOLOv10/v11/v12 families and RF-DETR for bounding-box detection, ensuring high-speed edge compatibility.

Dataset and Preprocessing Analysis

The system abstracts data ingestion by enforcing standardized structural requirements (folders for classification, COCO JSON for detection). Rather than applying a static, universal data augmentation policy, the preprocessing module dynamically constructs a sequential transformation pipeline. Base resizing and normalization are uniformly applied, while targeted augmentations (like simulated rain or motion blur) are injected specifically based on the linguistic context extracted by the LLM parser.

Evaluation Framework

The project evaluates success across two distinct vectors:

1. **Vision Quality Metrics:** Quantitative assessment of the output models using standard metrics including accuracy, macro-F1, mAP@0.5, and mAP@[0.5:0.95], alongside hardware profiling (FLOPs, parameter counts, and inference FPS).
2. **System Parsing Correctness:** A manual ablation rubric executed over twenty to thirty diverse prompts to verify that the LLM accurately translates complex linguistic constraints into the correct architectural and augmentative configurations without data loss.

Training and Deployment Strategy

The training strategy is defined by its strict adherence to a computational budget, managed dynamically by Ray Tune's resource allocation and early-stopping schedulers. The deployment strategy focuses on zero-configuration edge portability. By utilizing the ONNX format and embedding the required inference logic directly into the file's internal metadata properties, the system ensures the resulting artifact can be executed immediately on platforms like TensorRT without relying on external Python configuration scripts.

Schedule

The project is phased over a 12-week development cycle:

- **Weeks 1-4:** Development of the LLM parsing schemas, instructor integration, and CLIP-based label resolution.
- **Weeks 5-8:** Integration of the vision backbones and the construction of the budgeted Ray Tune/Optuna training loops.
- **Weeks 9-12:** Implementation of the ONNX metadata injection, execution of the evaluation ablation suite, and packaging of the final demonstration artifacts.

Risks and Failure Modes

- **LLM Configuration Hallucination:** Mitigated by the strict type-checking of pydantic schemas and automatic validation retry loops.
- **Label Mapping Mismatch:** Mitigated by enforcing strict cosine similarity thresholds during CLIP evaluation, falling back to substring matching if semantic alignment fails.
- **Budget Overruns during Training:** Mitigated by hard time-out parameters passed directly to the Ray Tune orchestrator.

Strengths

The primary strength of the Prompt2Model factory is its pragmatic, engineering-centric design. By leveraging proven, highly optimized open-source libraries (instructor, CLIP, Albumentations, YOLO, Ray Tune), the project avoids the pitfalls of designing novel neural architectures from scratch, focusing entirely on constructing a robust, automated MLOps pipeline.

Potential Gaps and Concerns

A notable engineering challenge remains in the calibration of the constraint extraction module. Accurately mapping subjective natural language phrases (e.g., "prioritize accuracy slightly over speed") into concrete numerical hyperparameter search boundaries for the Optuna backend will require significant empirical tuning to ensure the system behaves intuitively and produces optimal models.markdown

Prompt2Model: Language-Guided Vision

Model Factory

Team-Wide Execution Plan and Work Division

This document outlines the comprehensive 12-week execution strategy for the Prompt2Model system. To manage the multidisciplinary complexity of combining natural language processing, computer vision, and systems engineering, the workload is distributed across three distinct ownership domains, ensuring clear accountability for the project's authors: Dev Sanghvi, Venkata Sai Aneesh Thatiparti, and Madhuvani Thatiparti.

Ownership Domains

1. Dev Sanghvi — Language/Configuration Core & Labeling

Dev serves as the architect of the natural language frontend. The primary responsibility of this domain is to guarantee that the unstructured text provided by the user is translated flawlessly into a strict, programmatic format. This involves developing the pydantic schemas, managing the instructor LLM wrapper, engineering the mapping logic for data augmentations, and executing zero-shot label resolution using CLIP.

2. Venkata Sai Aneesh Thatiparti — Vision Data, Training & HPO

Venkata controls the core machine learning engine. This domain encompasses the integration of the computer vision dataset handlers, the curation and instantiation of the lightweight classification and detection backbones, and the orchestration of the computationally budgeted training loops utilizing Ray Tune and Optuna.

3. Madhuvani Thatiparti — Systems Integration, Export & Deployment

Madhuvani manages the terminal stages of the factory pipeline. This domain focuses on extracting the optimized PyTorch weights, exporting them into the ONNX format, injecting critical preprocessing rules into the model metadata, building the evaluation harness to calculate performance metrics, and ensuring the overall reproducibility of the codebase.

Individual Week-by-Week Deliverables

Dev Sanghvi (Language & Configuration Core)

Phase	Week	Deliverable Objective	Integration & Dependencies

Setup	1	Define the core pydantic schemas representing the hyperparameter search space, task definitions, and environmental constraints.	Coordinate with Venkata on available backbones.
	2	Implement the instructor LLM parser, configuring the system prompt and automatic retry logic for JSON validation failures.	Deliver base JSON configurations to unblock pipeline testing.
Mapping	3	Build the Albumentations mapping engine, translating parsed environmental strings (e.g., "rainy") into specific image transformations.	Validate transform compatibility with Venkata's dataloaders.
	4	Implement the zero-shot label resolution module using the Hugging Face transformers library and OpenAI's CLIP model.	Requires sample dataset annotation dictionaries from Venkata.

Refinement	5	Develop the assignment matrix algorithm for CLIP, utilizing cosine similarity thresholds to match user synonyms to dataset classes.	
	6	Construct the fallback logic for label resolution (e.g., substring matching) and tune the LLM temperature parameters.	
Integration	7	Execute the Pipeline Handshake; integrate the fully populated Configuration Object directly into the training orchestrator.	Direct handoff to Venkata.
	8	Debug dynamic augmentation crashes and edge-case prompt failures observed during end-to-end integration tests.	
Finalization	9-12	Run the Language-to-Config Correctness ablation	Supply parsing accuracy metrics to Madhuvani's evaluation

		suite across 30 distinct test prompts; draft corresponding sections of the final academic report.	report.

Venkata Sai Aneesh Thatiparti (Vision Training & HPO)

Phase	**Week**	**Deliverable Objective**	**Integration & Dependencies**
Setup	1	Develop standardized dataset loaders capable of ingesting classification directory structures and object detection COCO JSON files.	
	2	Integrate the constrained pool of vision backbones (e.g., MobileNetV3, YOLOv11, RF-DETR) into the PyTorch training environment.	
Engine	3	Establish the baseline training and validation loops, ensuring forward and backward passes execute correctly	

		without memory leaks.	
	4	Wire Dev's dynamic Albumentations transformation sequences into the PyTorch Dataset classes.	Requires the augmentation mapping output from Dev.
Budgeting	5	Integrate Optuna to define the continuous and discrete hyperparameter search spaces based on the selected backbones.	
	6	Implement Ray Tune orchestration, applying the Asynchronous Successive Halving Algorithm (ASHA) and strict maximum time limits to enforce the compute budget.	
Integration	7	Connect the Ray Tune launch scripts dynamically to the pydantic configuration schema generated by	Handoff the optimal model checkpoint weights to Madhuvani.

		the LLM parser.	
	8	Execute full pipeline stress tests on subset datasets to confirm that the early-stopping logic and time-budget halting function as intended.	
Finalization	9-12	Profile GPU-hour consumption across trials; finalize training loss logs; draft corresponding sections of the final academic report.	

Madhuvani Thatiparti (Systems Integration, Export & Deployment)

Phase	**Week**	**Deliverable Objective**	**Integration & Dependencies**
Setup	1	Initialize repository infrastructure, configuring CI/CD linting, and establishing strict environment pinning (requirements.txt).	

	2	Construct the standalone evaluation harness capable of calculating Accuracy, F1, and mAP@[0.5:0.95] from raw tensor outputs.	
Export	3	Develop the base structural export pipeline, converting dynamic PyTorch computational graphs into static ONNX formats.	Requires dummy model weights from Venkata.
	4	Implement ONNX graph manipulation utilities to parse and verify the exported artifact structures.	
Metadata	5	Build the metadata_props injection tool to embed target input resolutions, mean/std normalizations, and class dictionaries into the ONNX file.	Requires the finalized label mapping dictionaries from Dev.
	6	Develop a dummy edge-inference script utilizing ONNX	

		Runtime to verify the exported models can execute autonomously using only embedded metadata.	
Integration	7	Connect the ONNX metadata exporter to the terminal step of Venkata's Ray Tune training loop for automatic artifact generation.	
	8	Construct the end-to-end telemetry logger to benchmark host inference latency (FPS), parameter counts, and theoretical FLOPs.	
Finalization	9-12	Develop the script to generate the final formatted markdown Evaluation Report; package the reproducibility checks; prepare the final demonstration video.	

Shared Responsibilities

To ensure the cohesion of the automated pipeline, several critical functions are managed collectively:

- **Pull Request Reviews:** Code integration requires mandatory peer review; no code is merged into the main repository branch without approval from at least one adjacent domain owner.
- **End-to-End System Validation:** During Weeks 8 and 11, the entire team participates in holistic system testing, verifying that a natural language string accurately propagates through the LLM parser, surviving the budgeted training loop, and emerging as a valid ONNX artifact.
- **Documentation and Academic Reporting:** All authors are responsible for comprehensively documenting their respective modules and co-authoring the final 4-page academic project report.

RACI Matrix

This matrix establishes formal accountability for the major architectural components of the Prompt2Model factory. *(R = Responsible, A = Accountable, C = Consulted, I = Informed)*

System Component	Dev Sanghvi	Venkata S.A.T.	Madhuvani T.
LLM Constraint Parsing	R / A	I	I
Zero-Shot Label Resolution (CLIP)	R / A	C	I
Data Loaders & Augmentation Injection	C	R / A	I
Vision Backbone Integration	I	R / A	C

Budgeted HPO (Ray Tune / Optuna)	I	R / A	I
ONNX Export & Metadata Injection	C	C	R / A
Metrics & Evaluation Harness	I	C	R / A
Repository Operations & Reproducibility	I	C	R / A

Weekly Sync Structure and Checkpoints

To maintain alignment and rapidly address cross-domain blockers, the team adheres to a strict synchronization cadence:

- **Monday Sprint Planning (30 minutes):** Review the deliverables for the upcoming week and establish technical goals.
- **Wednesday Unblocker Check-in (15 minutes):** Asynchronous or brief synchronous meeting dedicated solely to resolving technical impediments preventing module integration.
- **Friday Integration Test (45 minutes):** Dedicated time to merge code and demonstrate functional module progress.

Critical Decision Checkpoints:

The following milestones require unanimous consent from all team members before development can proceed:

- **Week 2:** Final freeze of the pydantic configuration schema design, ensuring it supports all downstream requirements.
- **Week 4:** Finalization of the selected pool of vision backbones to prevent scope creep.
- **Week 7:** Official confirmation of the pipeline handshake, certifying that the prompt parser successfully communicates with the training orchestrator.

Prompt2Model: Language-Guided Vision Model Factory

Master Implementation Plan: Dev Sanghvi

Role: Language/Configuration Core & Labeling Lead

This document details the personal, phase-based execution roadmap for Dev Sanghvi. As the architect of the natural language frontend, the primary objective is to construct a deterministic, hallucination-free bridge that translates unstructured user intent into strictly typed parameters capable of orchestrating a computer vision training pipeline.

Phase 1: Structured Configuration Parsing (Weeks 1–3)

Focus: Designing the data structures that define the vision pipeline and coercing Large Language Models to output valid configurations.

- **Week 1: Pydantic Schema Architecture**
  - **Deliverable:** Develop pipeline_config.py encompassing the comprehensive pydantic data classes.
  - **Subcomponents:**
    - TaskType: Enum restricting choices to Classification or Detection.
    - ModelConstraints: Float representations of speed-vs-accuracy preferences, target inference latency, and maximum parameter size.
    - DataContext: Extraction of environmental descriptor strings.
    - ExtractedLabels: A list capturing user-defined target classes and their semantic synonyms.
  - **Validation Check:** Instantiate dummy configurations programmatically to ensure the schema successfully catches type mismatches and invalid Enum selections.
- **Week 2: Instructor LLM Integration**
  - **Deliverable:** Implement llm_parser.py utilizing the instructor library wrapped around the OpenAI or Anthropic base client.
  - **Subcomponents:** Engineer the system prompt to guide the LLM's analytical behavior. Configure the instructor validation loop to automatically retry generation upon encountering JSON schema violations.
  - **Validation Check:** Process ten diverse, edge-case natural language prompts through the parser. Verify the resulting pydantic objects accurately reflect the prompt's intent without introducing hallucinated variables.
- **Week 3: Albumentations Mapping Engine**
  - **Deliverable:** Construct augmentation_mapper.py to translate semantic environmental strings into executable transformation code.
  - **Subcomponents:** Develop a deterministic mapping dictionary linking keywords

(e.g., "poor lighting", "fast-moving objects") to specialized Albumentations operations (A.RandomBrightnessContrast, A.MotionBlur).
    - **Validation Check:** Serialize an augmentation pipeline based on a mock LLM output and verify it successfully transforms a dummy NumPy image array without throwing dimensional errors.

Phase 2: Zero-Shot Label Resolution (Weeks 4–6)

Focus: Automating the ontological alignment between the colloquial labels requested by the user and the rigid class names present in the target dataset's annotations.

- **Week 4: CLIP Feature Extraction**
    - **Deliverable:** Develop label_resolver.py integrating a pretrained Vision Transformer model (e.g., OpenAI CLIP ViT-B/32) using the Hugging Face transformers API.
    - **Subcomponents:** Implement the text encoder to generate dense semantic embeddings for both the LLM-parsed label synonyms and the unique class strings extracted from the raw dataset.
    - **Validation Check:** Confirm that tokenization and embedding operations execute rapidly and do not introduce GPU memory leaks.
- **Week 5: Cosine Similarity Matching**
    - **Deliverable:** Construct the mathematical assignment algorithm to bridge the semantic representations.
    - **Subcomponents:** Compute the cosine similarity matrix comparing the user label embeddings against the dataset label embeddings. Implement an argmax assignment algorithm to establish the definitive Dataset-to-User dictionary mapping.
    - **Validation Check:** Test the matrix against a controlled dummy dataset, ensuring that a user request for "motorcycle" correctly aligns with dataset annotations labeled "motorbike" and "two-wheeler".
- **Week 6: Fallback Logic and Threshold Tuning**
    - **Deliverable:** Finalize the error-handling logic for semantic mismatches.
    - **Subcomponents:** If the maximum cosine similarity fails to clear a predefined confidence threshold, the system must autonomously fallback to deterministic substring matching (e.g., Levenshtein distance).
    - **Validation Check:** Process intentionally mismatched edge cases (e.g., user asks for "aerospace vehicles" but dataset only contains "cats" and "dogs") to verify the fallback logic triggers correctly and logs the failure for the evaluation report.

Phase 3: Integration and Ablation (Weeks 7–12)

Focus: Seamlessly passing the validated configuration to the training engine and empirically proving the reliability of the language modules.

- **Week 7: Pipeline Handshake**
    - **Deliverable:** Integrate the outputs of llm_parser.py and label_resolver.py directly

into Venkata's training orchestrator.
    - **Validation Check:** Ensure the Ray Tune execution loop correctly receives and parses the serialized JSON dictionaries without type errors.
- **Week 8: Stabilization and Debugging**
    - **Deliverable:** Address cross-module integration bugs, specifically focusing on instances where dynamically mapped Albumentations transforms crash the PyTorch dataloaders during active training.
- **Weeks 9–10: Language-to-Config Ablation Suite**
    - **Deliverable:** Engineer and execute an automated evaluation script against a static repository of thirty complex test prompts.
    - **Validation Check:** Quantify the "Parsing Accuracy" metric by programmatically comparing the LLM's structural outputs against a manually verified ground-truth matrix, proving the system's reliability.
- **Weeks 11–12: Final Documentation**
    - **Deliverable:** Draft the sections of the final academic project report detailing the architecture of the Language-Guided Configurator, the pydantic constraint parsing, and the CLIP resolution mechanics.

Key Magic Numbers, Hyperparameters, and Tuning Knobs

To guarantee deterministic, reliable performance from the probabilistic language and vision models, the following parameters must be empirically tuned:

- **LLM Temperature:** Hardcoded to 0.1. A low temperature is mandatory to minimize creative hallucination, ensuring the language model focuses entirely on extracting factual constraints to populate the strict pydantic schema.
- **CLIP Cosine Similarity Threshold:** Initialized at 0.25. Values falling below this threshold mathematically indicate a lack of semantic correlation between the dataset ontology and the user's request, acting as the primary trigger for the fallback logic mechanism.
- **Maximum LLM Retries:** Set to 3. If the instructor wrapper fails to coerce the LLM into returning schema-compliant JSON after three consecutive attempts, the pipeline must throw a fatal validation error to prevent corrupted training runs.
- **Default Augmentation Probability (p):** Unless the natural language prompt explicitly demands heavy data corruption, the application probability for dynamically injected Albumentations transforms is locked at 0.5 to prevent catastrophic model divergence.

Final Summary Checklist

- [ ] The pydantic schema successfully maps the entirety of the vision hyperparameter and architectural search space.
- [ ] The instructor library consistently coerces the LLM into generating valid, hallucination-free JSON.
- [ ] The Albumentations mapping engine translates abstract text concepts into executable image transformations.

- [ ] The CLIP integration accurately resolves disparate label ontologies using cosine similarity matrices.
- [ ] The automated ablation suite empirically proves the language parsing module achieves >90% accuracy against the standardized 30-prompt rubric.markdown

Prompt2Model: Language-Guided Vision Model Factory

Artifact Walkthrough and Validation

What Was Done

This document package represents the systematic reverse-engineering and transformation of the high-level "Prompt2Model: Language-Guided Vision Model Factory" proposal into a comprehensive, implementation-ready execution framework. By utilizing the provided "Uncertainty-Calibrated Collision-Risk Forecasting" reference package as a structural and tonal template, this project translates the conceptual goal of natural language-driven Automated Machine Learning (AutoML) into a rigorous, actionable 12-week development blueprint. The analysis focuses explicitly on the architectural mechanics required to bridge large language models with strictly budgeted computer vision training pipelines.

Artifacts Created

The reverse-engineering process yielded five distinct documents, each tailored to a specific operational requirement of the project lifecycle:

1. **deep-research-report.md**
    - **Coverage:** Functions as the master architectural blueprint. It defines the precise operational scope, success metrics, and the end-to-end multi-agent pipeline. It catalogs the specific open-source libraries required for execution (e.g., instructor, pydantic, OpenAI CLIP, Albumentations, YOLOv11, and Ray Tune). Furthermore, it details the novel mechanism of embedding preprocessing configurations and label dictionaries directly into the metadata_props of the exported ONNX graph to satisfy the zero-configuration edge deployment requirement.
2. **paper_analysis.md**
    - **Coverage:** Provides a condensed, highly structured analysis of the system's design space. It summarizes the architecture deep-dive, highlights the strengths of utilizing LLMs to eliminate MLOps friction, and explicitly outlines the risks and mitigation strategies associated with autonomous hyperparameter tuning and potential prompt hallucination.
3. **work_division.md**
    - **Coverage:** A team-wide operational execution plan distributing the workload across the three specified authors (Dev Sanghvi, Venkata Sai Aneesh Thatiparti, Madhuvani

Thatiparti). It includes a week-by-week deliverable timeline, establishes formal accountability via a RACI matrix, and defines the critical integration handoff points and weekly sync structures necessary for team cohesion.

4. **dev_work_division.md**
    - **Coverage:** A granular, personal implementation plan specifically constructed for Dev Sanghvi (Lead for Language/Configuration Core). It breaks down his domain into four chronological phases, detailing specific engineering tasks (such as implementing the CLIP cosine similarity matrix and the Albumentations mapping engine), while explicitly defining validation checks and tuning hyperparameters.
5. **walkthrough.md**
    - **Coverage:** This meta-document. It summarizes the purpose, creation, and validation of the entire project package, ensuring all constraints of the original request have been met.

Key Findings

During the expansion of the project proposal into a technical blueprint, several critical engineering insights were identified:

- **The Necessity of Structured LLM Parsing:** Relying on raw text outputs from large language models is fundamentally incompatible with the rigid requirements of a machine learning training orchestrator. The integration of libraries like instructor and pydantic is not optional; it is a mandatory architectural requirement to enforce strict JSON schemas and prevent catastrophic pipeline failures due to hallucination.
- **Zero-Shot Ontology Bridging:** Automating a vision pipeline requires an autonomous method to map colloquial user terminology to rigid dataset labels. The application of OpenAI's CLIP model to perform zero-shot cosine similarity matching effectively solves this bottleneck, eliminating the need for human-in-the-loop dictionary mapping.
- **ONNX Metadata for True Portability:** Exporting neural weights alone fails the primary objective of a "model factory" intended for edge deployment. Embedding preprocessing parameters (like normalization statistics and input resolutions) directly into the ONNX graph's metadata_props is the critical mechanism that allows the resulting artifact to be deployed autonomously on platforms like TensorRT.

Validation

The generated artifacts adhere to the following constraints and quality bars:

- **Scope Compliance:** All project-specific content and architectural design choices were derived exclusively from the main.pdf proposal, augmented by current industry-standard practices (e.g., YOLOv11 for NMS-free detection, Ray Tune with ASHA for budgeted tuning). No domain content from the collision-risk reference package was copied.
- **Budgetary Realism:** The project architecture respects the requested "modest compute budget" by mandating the use of pretrained lightweight backbones and employing aggressive early-stopping schedulers in Optuna to prevent resource overruns.

- **Structural Rigor:** The output maintains an expert, implementation-first persona. It utilizes narrative prose for qualitative reasoning and reserves markdown tables strictly for structured data, fulfilling the formatting constraints while delivering maximum technical density.

Works cited

1. IMPROVE: Iterative Model Pipeline Refinement and Optimization Leveraging LLM Agents, accessed April 1, 2026, https://arxiv.org/html/2502.18530v1
2. A Multi-Agent LLM Framework for Full-Pipeline AutoML - ICML 2026, accessed April 1, 2026, https://icml.cc/virtual/2025/poster/44029
3. AutoMMLab: Automatically Generating Deployable Models from Language Instructions for Computer Vision Tasks - AAAI Publications, accessed April 1, 2026, https://ojs.aaai.org/index.php/AAAI/article/view/34358/36513
4. Lightweight Transformer Architectures for Edge Devices in Real-Time Applications - arXiv, accessed April 1, 2026, https://arxiv.org/html/2601.03290v1
5. Prompt2Model: Generating Deployable Models from Natural Language Instructions - arXiv, accessed April 1, 2026, https://arxiv.org/abs/2308.12261
6. Prompt2Model: Generating Deployable Models from Natural Language Instructions, accessed April 1, 2026, https://aclanthology.org/2023.emnlp-demo.38/
7. prompt2model - Generate Deployable Models from Natural Language Instructions - GitHub, accessed April 1, 2026, https://github.com/neulab/prompt2model
8. A Multi-Agent LLM Framework for Full-Pipeline AutoML - arXiv, accessed April 1, 2026, https://arxiv.org/pdf/2410.02958
9. Local inference using ONNX for AutoML image - Azure Machine Learning, accessed April 1, 2026, https://docs.azure.cn/en-us/machine-learning/how-to-inference-onnx-automl-image-models?view=azureml-api-2
10. Run ONNX inference in WebAssembly data flow graphs - Azure IoT Operations, accessed April 1, 2026, https://learn.microsoft.com/en-us/azure/iot-operations/develop-edge-apps/howto-wasm-onnx-inference
11. How to Deploy AI/ML Models at the Edge with K3s - OneUptime, accessed April 1, 2026, https://oneuptime.com/blog/post/2026-03-20-k3s-aiml-edge-deployment/view
12. Set up AutoML for computer vision (v1) - Azure Machine Learning, accessed April 1, 2026, https://learn.microsoft.com/en-au/Azure/machine-learning/how-to-auto-train-image-models?view=azureml-api-1
13. AutoML Vision: Complete Guide to Google Cloud Vision AI & Vertex AI Image Models, accessed April 1, 2026, https://www.philhardin.org/discover.html?app=build-custom-image-models-with-automl-vision-in-hours-69ae4133a3b10

14. ENVI Deep Learning Tutorial: Configure ONNX Model - NV5 Geospatial Software, accessed April 1, 2026, https://www.nv5geospatialsoftware.com/docs/ENVIDeepLearningTutorialConfigureONNX.html
15. From Prompt to Production: What We Learned from the AutoML-Agent Framework | by Kai Ni, accessed April 1, 2026, https://medium.com/@kai.ni/from-prompt-to-production-what-we-learned-from-the-automl-agent-framework-b7e792fc0cbe
16. Structured Data Extraction using instructor | Modal Docs, accessed April 1, 2026, https://modal.com/docs/examples/instructor_generate
17. 567-labs/instructor: structured outputs for llms - GitHub, accessed April 1, 2026, https://github.com/567-labs/instructor
18. Your First LLM Extraction: Structured Outputs Tutorial - Instructor, accessed April 1, 2026, https://python.useinstructor.com/learning/getting_started/first_extraction/
19. DELM: a Python toolkit for Data Extraction with Language Models - arXiv, accessed April 1, 2026, https://arxiv.org/html/2509.20617v1
20. CHiLS: Zero-Shot Image Classification with Hierarchical Label Sets - Proceedings of Machine Learning Research, accessed April 1, 2026, https://proceedings.mlr.press/v202/novack23a/novack23a.pdf
21. Zero-shot Image Classification with OpenAI's CLIP - Pinecone, accessed April 1, 2026, https://www.pinecone.io/learn/series/image-search/zero-shot-image-classification-clip/
22. Results Of CLIP's Zero-Shot Performance Over OCR Dataset - Labellerr, accessed April 1, 2026, https://www.labellerr.com/blog/zero-shot-clip-model-ocr/
23. Auto-Label Classification Datasets Using CLIP - Roboflow Blog, accessed April 1, 2026, https://blog.roboflow.com/how-to-auto-label-classification-datasets/
24. CLIP: Creating Image Classifiers Without Data | by Lihi Gur Arie, PhD - Medium, accessed April 1, 2026, https://medium.com/data-science/clip-creating-image-classifiers-without-data-b21c72b741fa
25. Albumentations: fast and flexible image augmentations, accessed April 1, 2026, https://albumentations.ai/
26. How to Use Albumentations for Computer Vision: Step By Step - Roboflow Blog, accessed April 1, 2026, https://blog.roboflow.com/how-to-use-albumentations/
27. Image Augmentation for Machine Learning: Techniques, Examples & Code | Datature Blog, accessed April 1, 2026, https://datature.io/blog/image-augmentation-for-machine-learning-techniques-examples-code
28. Enhance Your Dataset to Train YOLO26 Using Albumentations - Ultralytics YOLO Docs, accessed April 1, 2026, https://docs.ultralytics.com/integrations/albumentations/
29. Running Tune experiments with Optuna - Ray Docs, accessed April 1, 2026, https://docs.ray.io/en/latest/tune/examples/optuna_example.html

30. ray.tune.search.optuna.OptunaSearch — Ray 2.54.1 - Ray Docs, accessed April 1, 2026, https://docs.ray.io/en/latest/tune/api/doc/ray.tune.search.optuna.OptunaSearch.html
31. Efficient Hyperparameter Tuning with Ray Tune and YOLO26 - Ultralytics YOLO Docs, accessed April 1, 2026, https://docs.ultralytics.com/integrations/ray-tune/
32. ONNX Concepts - ONNX 1.22.0 documentation, accessed April 1, 2026, https://onnx.ai/onnx/intro/concepts.html
33. Instructor - Multi-Language Library for Structured LLM Outputs | Python, TypeScript, Go, Ruby - Instructor, accessed April 1, 2026, https://python.useinstructor.com/
34. How to Use Pydantic for LLMs: Schema, Validation & Prompts, accessed April 1, 2026, https://pydantic.dev/articles/llm-intro
35. CLIP-driven Zero-shot Learning with Ambiguous Labels - arXiv, accessed April 1, 2026, https://arxiv.org/html/2603.05053v1
36. CLIP: Teaching Vision Models to Understand Natural Language | by Dong-Keon Kim, accessed April 1, 2026, https://medium.com/@kdk199604/clip-teaching-vision-models-to-understand-natural-language-0eeceebdcf3c
37. Augmenting Datasets with Albumentations - FiftyOne - Voxel51, accessed April 1, 2026, https://docs.voxel51.com/tutorials/data_augmentation.html
38. Data Augmentation Techniques in Computer Vision and NLP: A Complete Guide - Medium, accessed April 1, 2026, https://medium.com/@vamsikd219/data-augmentation-techniques-in-computer-vision-and-nlp-a-complete-guide-6cbbeabc3a53
39. Edge AI Vision: Deep Learning on Tiny Devices - Medium, accessed April 1, 2026, https://medium.com/@API4AI/edge-ai-vision-deep-learning-on-tiny-devices-11382f327db6
40. Edge-Optimized Lightweight YOLO for Real-Time SAR Object Detection - MDPI, accessed April 1, 2026, https://www.mdpi.com/2072-4292/17/13/2168
41. Best Object Detection Models 2025: RF-DETR, YOLOv12 & Beyond - Roboflow Blog, accessed April 1, 2026, https://blog.roboflow.com/best-object-detection-models/
42. Top Object Detection Models for Your Projects in 2025 | DigitalOcean, accessed April 1, 2026, https://www.digitalocean.com/community/tutorials/best-object-detection-models-guide
43. ELNet: An Efficient and Lightweight Network for Small Object Detection in UAV Imagery, accessed April 1, 2026, https://www.mdpi.com/2072-4292/17/12/2096
44. Optuna - A hyperparameter optimization framework, accessed April 1, 2026, https://optuna.org/
45. Local inference using ONNX for AutoML image - Azure Machine Learning | Microsoft Learn, accessed April 1, 2026, https://learn.microsoft.com/en-us/azure/machine-learning/how-to-inference-onnx-automl-image-models?view=azureml-api-2

46. What Is Image Augmentation? - Albumentations, accessed April 1, 2026, https://albumentations.ai/docs/1-introduction/what-are-image-augmentations/
47. Albumentations: Fast and Flexible Image Augmentations - MDPI, accessed April 1, 2026, https://www.mdpi.com/2078-2489/11/2/125
48. Ray Tune: Hyperparameter Tuning — Ray 2.54.1 - Ray Docs, accessed April 1, 2026, https://docs.ray.io/en/latest/tune/index.html
49. AutoMMLab: Automatically Generating Deployable Models from Language Instructions for Computer Vision Tasks - arXiv, accessed April 1, 2026, https://arxiv.org/html/2402.15351v1
50. Improving Accessibility for the Visually Impaired with Enhanced Multi-class Object Detection Using YOLOv8L, YOLO11x, and Faster R-CNN with Diverse Backbones - ScienceOpen, accessed April 1, 2026, https://www.scienceopen.com/hosted-document?doi=10.57197/JDR-2025-0642
51. Advancements in Small-Object Detection (2023–2025): Approaches, Datasets, Benchmarks, Applications, and Practical Guidance - MDPI, accessed April 1, 2026, https://www.mdpi.com/2076-3417/15/22/11882
52. How Do Model Export Formats Impact the Development of ML-Enabled Systems? A Case Study on Model Integration - arXiv, accessed April 1, 2026, https://arxiv.org/html/2502.00429v1
53. Metadata - Python API documentation - ONNX Runtime, accessed April 1, 2026, https://onnxruntime.ai/docs/api/python/auto_examples/plot_metadata.html
54. JSON Metadata Parameters for ONNX Models - Oracle Help Center, accessed April 1, 2026, https://docs.oracle.com/en/database/oracle/oracle-database/26/vecse/json-metadata-parameters-onnx-models.html
55. Accelerating LLM and VLM Inference for Automotive and Robotics with NVIDIA TensorRT Edge-LLM | NVIDIA Technical Blog, accessed April 1, 2026, https://developer.nvidia.com/blog/accelerating-llm-and-vlm-inference-for-automotive-and-robotics-with-nvidia-tensorrt-edge-llm/
56. ZERO-SHOT PSEUDO LABELS GENERATION USING SAM AND CLIP FOR SEMI-SUPERVISED SEMANTIC SEGMENTATION - GitHub Pages, accessed April 1, 2026, https://gsisaoki.github.io/ZERO-SHOT-PLG/
57. mlfoundations/open_clip: An open source implementation of CLIP. - GitHub, accessed April 1, 2026, https://github.com/mlfoundations/open_clip
58. Learning Transferable Visual Models From Natural Language Supervision - arXiv, accessed April 1, 2026, https://arxiv.org/pdf/2103.00020
59. albumentations-team/albumentations: Fast and flexible image augmentation library. Paper about the library: https://www.mdpi.com/2078-2489/11/2/125 · GitHub - GitHub, accessed April 1, 2026, https://github.com/albumentations-team/albumentations